Sistem Pakar Rekomendasi Berbasis Ripple Down Rules (RDR)

Laporan Tugas <Nama Mata Kuliah>

<Nama Anggota 1> (13523065)
<Nama Anggota 2> (13523117)

31 Oktober 2026

Daftar Isi

1 Pendahuluan **3**
1.1 Deskripsi Tugas . . . 3
1.2 Ringkasan Solusi . . . 3

2 Dasar Teori **3**
2.1 Ripple Down Rules . . . 3
2.2 Struktur Single Classification RDR . . . 3
2.3 Inferensi . . . 4
2.4 Akuisisi Pengetahuan dan Difference List . . . 4

3 Analisis dan Desain **4**
3.1 Arsitektur . . . 4
3.2 Struktur Data . . . 5
3.3 Konfigurasi Domain . . . 5
3.4 Keputusan Desain dan Asumsi . . . 5

4 Algoritma **7**

5 Implementasi **8**
5.1 Struktur Proyek . . . 8
5.2 Antarmuka Web (Streamlit) . . . 9
5.3 Mode Terminal (CLI) . . . 10
5.4 Cara Menjalankan . . . 10

6 Pengujian **11**
6.1 Domain Uji . . . 11

6.2 Skenario Input dan Output 11
6.3 Pembentukan Rule 11
6.4 Tree Akhir 12
6.5 Uji Regresi Cornerstone 13
6.6 Penanganan Conflict 13
6.7 Edge Case 13
6.8 Pengujian Otomatis 14

7 Kesimpulan dan Saran **14**
7.1 Kesimpulan 14
7.2 Saran 14

8 Pembagian Kerja **14**

1 Pendahuluan

1.1 Deskripsi Tugas

Tugas ini meminta pembuatan sistem AI berbasis *Ripple Down Rules* (RDR) yang dipakai seorang pakar untuk memberi rekomendasi kepada konsumen. Ketentuan utamanya:

1. Knowledge base (KB) awal hanya berisi satu rule, $t \rightarrow$ kesimpulan default, yang dapat diisi sesuai domain masalah.
2. Kesimpulan diperoleh dengan menelusuri binary tree. Anak kanan (*true-child*) adalah pengecualian yang dieksekusi jika kondisi node terpenuhi; anak kiri (*false-child*) dieksekusi jika kondisi tidak terpenuhi.
3. Kesimpulan yang diberikan adalah kesimpulan terakhir yang diperoleh selama penelusuran.
4. Jika pakar setuju, kesimpulan diberikan dan proses selesai. Jika tidak, dibuat rule pengecualian baru dengan kesimpulan dari pakar. Kondisinya dipilih dari kondisi yang dipenuhi kasus baru tetapi tidak dipenuhi kasus kunci (*cornerstone case*) dari kesimpulan yang dianulir.
5. Atribut dapat berjenis numerik maupun kategorikal.

1.2 Ringkasan Solusi

Kami membangun engine *Single Classification RDR* (SCRDR) yang generik terhadap domain. Domain (nama, kesimpulan default, dan daftar atribut) dibaca dari file konfigurasi, sehingga sistem yang sama dapat dipakai untuk domain apa pun. Engine dilengkapi dua antarmuka: aplikasi web lokal (Streamlit) dengan visualisasi tree, dan mode terminal (CLI) yang dapat merekam sesi ke file teks. Domain contoh yang dipakai pada laporan ini adalah *rekomendasi produk keuangan.*

2 Dasar Teori

2.1 Ripple Down Rules

Ripple Down Rules adalah metode akuisisi pengetahuan yang diperkenalkan oleh Compton dan Jansen. Alih-alih meminta pakar menyusun seluruh basis pengetahuan di awal, pengetahuan ditambahkan secara bertahap ketika sistem salah menangani suatu kasus. Pakar cukup menjelaskan *apa yang membedakan* kasus baru dari kasus lama yang menjadi alasan rule sebelumnya dibuat. Pengetahuan lama tidak pernah diubah atau dihapus, hanya diberi pengecualian.

2.2 Struktur Single Classification RDR

Pada SCRDR, KB berbentuk binary tree. Setiap node menyimpan:

- **kondisi**: konjungsi kondisi atomik (root memiliki kondisi *true*);
- **kesimpulan**;
- **cornerstone case**: kasus yang menyebabkan rule tersebut dibuat;
- **true-child** (kanan): pengecualian, dikunjungi jika kondisi TRUE;
- **false-child** (kiri): alternatif, dikunjungi jika kondisi FALSE.

Karena kondisi root selalu TRUE, false-child root tidak pernah dikunjungi. Seluruh rule yang menganulir kesimpulan default membentuk rantai false-child di bawah true-child root, yang

dapat dibaca sebagai "default, *kecuali* $a \rightarrow X$, *atau jika tidak* $e \rightarrow Y$, ...".

2.3 Inferensi

Penelusuran dimulai dari root. Jika kondisi node terpenuhi, kesimpulannya dicatat lalu penelusuran berlanjut ke true-child; jika tidak, ke false-child. Penelusuran berhenti saat child yang dituju kosong. Kesimpulan akhir adalah kesimpulan node terakhir yang bernilai TRUE.

2.4 Akuisisi Pengetahuan dan Difference List

Saat pakar tidak setuju, dua node dari hasil penelusuran menjadi penting:

- **lastTrue**: node terakhir bernilai TRUE, yaitu pemberi kesimpulan yang salah. *Cornerstone* node ini menjadi pembanding.
- **lastEval**: node terakhir yang dievaluasi. Node baru ditempel di sini, sebagai true-child jika kondisinya TRUE atau false-child jika FALSE.

Difference list adalah himpunan kondisi yang bernilai TRUE untuk kasus baru dan FALSE untuk cornerstone **lastTrue**. Dengan memilih kondisi dari daftar ini, cornerstone lama dijamin tetap mendapat kesimpulan lamanya, sehingga tidak terjadi regresi.

Cornerstone **lastTrue** selalu mencapai posisi node baru: semua pengecualian **lastTrue** yang sudah ada dibuat agar FALSE untuk cornerstone tersebut, sehingga cornerstone itu menelusuri rantai pengecualian sampai ujung. Karena itu cukup cornerstone ini saja yang perlu dibedakan.

3 Analisis dan Desain

3.1 Arsitektur

Sistem dibagi menjadi tiga lapis yang terpisah:

1. **Engine** (`rdr/`): struktur data, inferensi, difference list, penambahan rule, validasi, serta penyimpanan. Engine tidak bergantung pada antarmuka apa pun.
2. **Antarmuka**: aplikasi web Streamlit (`app.py`, `ui/`) dan mode terminal (`cli.py`).
3. **Data**: satu folder per domain di `domains/`, berisi `domain.json` (ditulis manusia) dan `kb.json` (ditulis sistem).

3.2 Struktur Data

Struktur	Isi
`Case`	Pemetaan nama atribut ke nilai (float untuk numerik, string untuk kategorikal).
`Condition`	Tripel (atribut, operator, nilai). Operator numerik: `>`, `>=`, `<`, `<=`. Operator kategorikal: `==`, `!=`. Atribut yang tidak ada pada kasus membuat kondisi bernilai FALSE.
`Node`	id, daftar kondisi (AND), kesimpulan, cornerstone, true-child, false-child, serta metadata posisi (parent, cabang) dan id node yang dianulir.
`KnowledgeBase`	Root dan penghitung id; operasi `infer`, `add_rule`, `difference_list`, `verify`.
`Domain`	Nama, kesimpulan default, dan daftar `Attribute` (nama, tipe, daftar nilai opsional, min/max/satuan opsional).

Tabel 1: Struktur data utama

3.3 Konfigurasi Domain

Domain didefinisikan oleh `domain.json`. Informasi minimal yang diperlukan adalah kesimpulan default serta nama dan tipe setiap atribut. Tipe atribut diperlukan untuk menentukan operator kandidat dan cara membandingkan nilai.

```
{
  "name": "Rekomendasi Produk Keuangan",
  "default_conclusion": "Tabungan Reguler",
  "attributes": {
    "umur":   {"type": "numeric", "min": 17, "max": 100, "unit": "tahun"},
    "gaji":   {"type": "numeric", "min": 0, "unit": "juta/bulan"},
    "risiko": {"type": "categorical", "values": ["rendah", "sedang", "tinggi"]},
    "status": {"type": "categorical", "values": ["pelajar", "pekerja", "pensiun"]}
  }
}
```

Domain baru dapat dibuat lewat antarmuka (formulir di Streamlit atau wizard di CLI) maupun dengan menulis `domain.json` secara manual. Atribut baru dapat ditambahkan kapan saja.

3.4 Keputusan Desain dan Asumsi

Beberapa hal tidak dirinci oleh spesifikasi. Tabel 2 merangkum keputusan kami.

Aspek	Keputusan
Jumlah kondisi per rule	Boleh lebih dari satu (digabung AND). Setiap kondisi harus berasal dari difference list.
Pemilih kondisi	Pakar memilih dari difference list yang ditampilkan sistem. Sistem memvalidasi pilihan.

Aspek	**Keputusan**
Kandidat kondisi numerik	Relatif terhadap cornerstone. Jika nilai kasus baru $v > c$ (nilai cornerstone): `attr > c` (paling umum) dan `attr >= v` (paling spesifik). Jika $v < c$: `attr < c` dan `attr <= v`. Jika $v = c$: tidak ada kandidat. Tanpa cornerstone: `attr >= v` dan `attr <= v`.
Kandidat kondisi kategorikal	`attr == v`, ditambah `attr != c` jika nilai cornerstone berbeda.
Threshold kustom	Pakar boleh memasukkan kondisi sendiri (misalnya `umur > 45`) selama kondisi itu TRUE untuk kasus baru dan FALSE untuk cornerstone.
Cornerstone root	Root tidak memiliki cornerstone, sehingga rule yang menganulir default boleh memakai kondisi apa pun dari kasus baru.
Difference list kosong	Disebut *conflict*. Rule lama tidak diubah. Pakar dapat membatalkan koreksi atau menambah atribut baru yang membedakan kedua kasus.
Atribut baru	Cornerstone lama tidak memiliki nilai atribut tersebut, sehingga kondisi pada atribut itu bernilai FALSE untuk cornerstone lama.
Nilai kosong	Semua atribut wajib diisi saat konsultasi.
Kesimpulan sama	Jika pakar memberi kesimpulan yang sama dengan sistem, dianggap setuju dan tidak ada rule baru.
Penulisan kesimpulan	Dinormalisasi (spasi, huruf besar/kecil) mengikuti kesimpulan yang sudah ada, misalnya “saham” menjadi “Saham”.
Persistensi	KB disimpan otomatis ke `kb.json` setiap kali rule ditambahkan. Reset membuat backup file lama.

Tabel 2: Keputusan desain dan asumsi

4 Algoritma

Algoritma 1 Inferensi

Masukan: root KB, kasus x
Keluaran: kesimpulan, lastTrue, lastEval, lastResult, path

$node \leftarrow root$; $lastTrue \leftarrow root$; $lastEval \leftarrow root$
$lastResult \leftarrow$ **true**; $path \leftarrow [\,]$
while $node \neq$ **null do**
    $lastEval \leftarrow node$
    if semua kondisi $node$ dipenuhi x **then**
        $lastTrue \leftarrow node$; $lastResult \leftarrow$ **true**
        tambahkan $(node,$ **true**$)$ ke $path$
        $node \leftarrow node.trueChild$ ▷ anak kanan: pengecualian
    else
        $lastResult \leftarrow$ **false**
        tambahkan $(node,$ **false**$)$ ke $path$
        $node \leftarrow node.falseChild$ ▷ anak kiri: alternatif
    end if
end while
return $(lastTrue.conclusion, lastTrue, lastEval, lastResult, path)$

Algoritma 2 Pembangkitan Difference List

Masukan: kasus baru x, cornerstone c (boleh **null**), domain
Keluaran: daftar kondisi pembeda

$cand \leftarrow [\,]$
for setiap atribut a pada domain **do**
    $v \leftarrow x[a]$; $w \leftarrow c[a]$ jika ada, selain itu **null**
    if a numerik **then**
        if $w =$ **null then** tambahkan $a \geq v$, $a \leq v$
        else if $v > w$ **then** tambahkan $a > w$, $a \geq v$
        else if $v < w$ **then** tambahkan $a < w$, $a \leq v$
        end if
    else
        tambahkan $a = v$
        if $w \neq$ **null and** $w \neq v$ **then** tambahkan $a \neq w$
        end if
    end if
end for
return $[\, k \in cand \mid k(x) =$ **true** $\wedge\, k(c) =$ **false** $]$

Algoritma 3 Penambahan Rule

Masukan: KB, kasus x, kesimpulan pakar k, kondisi terpilih C

$(concl, lastTrue, lastEval, lastResult, _) \leftarrow$ INFER$(root, x)$
if $k = concl$ **then tolak**: kesimpulan sama
end if
if C kosong **then tolak**: minimal satu kondisi
end if
for setiap $q \in C$ **do**
    if $q(x) =$ **false then tolak**: tidak dipenuhi kasus baru
    end if
    if $q(lastTrue.cornerstone) =$ **true then tolak**: cornerstone ikut teranulir
    end if
end for
$n \leftarrow$ NODE$(C, k, cornerstone =$ salinan$(x))$
if $lastResult$ **then**
    $lastEval.trueChild \leftarrow n$
else
    $lastEval.falseChild \leftarrow n$
end if
simpan KB

Algoritma 4 Siklus Konsultasi

muat domain dan KB (jika belum ada: $root \leftarrow$ IF true THEN default)
loop
    $x \leftarrow$ input data konsumen (divalidasi sesuai tipe atribut)
    $res \leftarrow$ INFER$(root, x)$; tampilkan kesimpulan dan jalur
    if pakar setuju **then**
        berikan $res.conclusion$ kepada konsumen
    else
        $k \leftarrow$ kesimpulan yang benar dari pakar
        $D \leftarrow$ DIFFERENCELIST$(x, res.lastTrue.cornerstone)$
        while D kosong **do** ▷ conflict
            pakar membatalkan, **atau** menambah atribut baru dan nilainya untuk x
            $D \leftarrow$ DIFFERENCELIST$(x, res.lastTrue.cornerstone)$
        end while
        $C \leftarrow$ kondisi yang dipilih pakar dari D
        ADDRULE(x, k, C); berikan k kepada konsumen
    end if
end loop

5 Implementasi

5.1 Struktur Proyek

```
app.py              entry point Streamlit
cli.py              mode terminal
rdr/                engine (tidak bergantung pada UI)
  models.py         Condition, Node
  engine.py         KnowledgeBase: infer(), add_rule(), verify()
  difference.py     kandidat kondisi dan difference list
```

```
  schema.py          Domain, Attribute, validasi kasus, load/save domain.json
  storage.py         load/save/reset kb.json
ui/                  halaman Streamlit
domains/             satu folder per domain (domain.json, kb.json)
scripts/             skenario demo
tests/               pytest
```

5.2 Antarmuka Web (Streamlit)

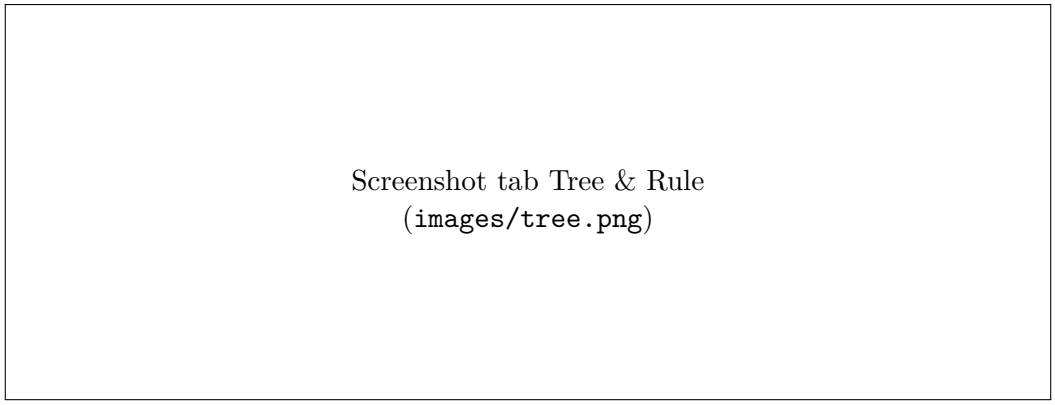
Aplikasi web terdiri dari empat tab:

- **Konsultasi**: formulir data konsumen yang dibangun otomatis dari schema domain. Menampilkan rekomendasi, tabel jalur penelusuran, dan posisi kasus pada tree. Dalam mode pakar tersedia tombol setuju/tidak setuju, pilihan kesimpulan, tabel perbandingan kasus baru dengan cornerstone, difference list, kondisi kustom, pratinjau rule beserta posisinya, dan penanganan conflict.
- **Tree & Rule**: visualisasi binary tree (anak kiri F berwarna merah, anak kanan T berwarna hijau, jalur kasus terakhir disorot), daftar rule beserta cornerstone, verifikasi KB, unduh `kb.json`, dan reset KB.
- **Uji Batch (CSV)**: menjalankan inferensi untuk banyak kasus sekaligus.
- **Domain & Atribut**: melihat schema, mengubah default, menambah atribut, dan membuat domain baru.

Sidebar berisi pemilihan domain dan toggle *mode pakar*. Ketika toggle dimatikan, sistem berperan sebagai mode konsumen yang hanya menampilkan rekomendasi.



Gambar 1: Tab Konsultasi dalam mode pakar



Gambar 2: Visualisasi tree pada tab Tree & Rule

5.3 Mode Terminal (CLI)

Mode terminal menyediakan alur yang sama melalui menu teks: konsultasi, koreksi pakar, difference list bernomor, kondisi kustom, penanganan conflict, tree ASCII, daftar rule, verifikasi, penambahan atribut, serta pembuatan domain. Opsi `-log` merekam seluruh prompt dan jawaban ke file teks. Potongan rekaman sesi:

```
Data konsumen:
  umur (angka, 17..100, tahun): 60
  gaji (angka, 0..inf, juta/bulan): 20
  risiko (rendah/sedang/tinggi): tinggi
  status (pelajar/pekerja/pensiun): pensiun

Jalur penelusuran:
  R0: IF true THEN Tabungan Reguler        -> TRUE (ke anak kanan)
  R1: IF risiko == tinggi THEN Saham       -> TRUE (ke anak kanan)

REKOMENDASI: Saham   (dari R1)
Pakar setuju? [y/n]: n
Kesimpulan yang benar (ketik nomor atau teks baru, kosong = batal):
  1. Tabungan Reguler
> Deposito

Kesimpulan 'Saham' (R1) akan dianulir.
  Atribut | Kasus baru | Cornerstone R1
  --------+------------+---------------
  umur    | 60         | 30
  gaji    | 20         | 15
  risiko  | tinggi     | tinggi
  status  | pensiun    | pekerja

Difference list:
  1. umur > 30
  2. umur >= 60
  3. gaji > 15
  4. gaji >= 20
  5. status == pensiun
  6. status != pekerja
> 5
  Kondisi: status == pensiun
  Sudah cukup? [y/n]: y

Rule R2 ditambahkan: IF status == pensiun THEN Deposito
Ditempel di R1.trueChild, pengecualian dari R1.
```

5.4 Cara Menjalankan

```
pip install -r requirements.txt
streamlit run app.py                                  # aplikasi web
python cli.py --domain keuangan                       # mode terminal (pakar)
python cli.py --domain keuangan --consumer            # mode konsumen
python cli.py --domain keuangan --log docs/sesi.txt   # rekam sesi
python -m scripts.demo_scenario                       # jalankan skenario C1-C9
python -m pytest                                      # pengujian
```

6 Pengujian

6.1 Domain Uji

Domain rekomendasi produk keuangan memiliki kesimpulan default *Tabungan Reguler* dan empat atribut: `umur` (numerik, tahun), `gaji` (numerik, juta/bulan), `risiko` (kategorikal: rendah, sedang, tinggi), dan `status` (kategorikal: pelajar, pekerja, pensiun). Pengujian dimulai dari KB yang hanya berisi $R0$: IF true THEN Tabungan Reguler.

6.2 Skenario Input dan Output

Tabel 3 menunjukkan sembilan kasus yang dimasukkan secara berurutan. Notasi R_i(T) berarti kondisi R_i terpenuhi sehingga penelusuran berlanjut ke anak kanan; R_i(F) berarti tidak terpenuhi sehingga berlanjut ke anak kiri.

#	**Input (umur, gaji, risiko, status)**	**Jalur**	**Output sistem**	**Pakar**	**Aksi**
C1	20, 2, rendah, pelajar	R0(T)	Tabungan Reguler	Setuju	–
C2	30, 15, tinggi, pekerja	R0(T)	Tabungan Reguler	Saham	R1 di R0.trueChild
C3	25, 8, sedang, pekerja	R0(T), R1(F)	Tabungan Reguler	Reksadana	R2 di R1.falseChild
C4	60, 20, tinggi, pensiun	R0(T), R1(T)	Saham	Deposito	R3 di R1.trueChild
C5	35, 25, tinggi, pekerja	R0(T), R1(T), R3(F)	Saham	Setuju	–
C6	22, 3, tinggi, pelajar	R0(T), R1(T), R3(F)	Saham	Reksadana	R4 di R3.falseChild
C7	65, 2, tinggi, pensiun	R0(T), R1(T), R3(T)	Deposito	Setuju	– (R4 tidak dicek walau `gaji <= 3` benar)
C8	50, 40, sedang, pekerja	R0(T), R1(F), R2(T)	Reksadana	Obligasi	R5 di R2.trueChild
C9	30, 15, tinggi, pekerja	R0(T), R1(T), R3(F), R4(F)	Saham	Obligasi	**Conflict**: difference list kosong

Tabel 3: Skenario pengujian C1–C9

6.3 Pembentukan Rule

Tabel 4 menunjukkan difference list yang dihasilkan sistem untuk setiap koreksi dan kondisi yang dipilih pakar.

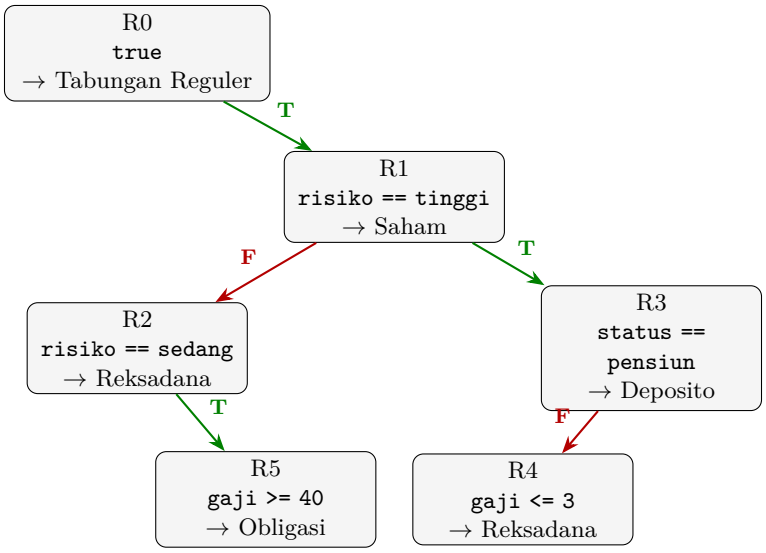
Rule	Kasus	Pembanding	Difference list	Rule terbentuk
R1	C2	R0 (tanpa cornerstone)	`umur >= 30`, `umur <= 30`, `gaji >= 15`, `gaji <= 15`, `risiko == tinggi`, `status == pekerja`	IF `risiko == tinggi` THEN Saham
R2	C3	R0 (tanpa cornerstone)	`umur >= 25`, `umur <= 25`, `gaji >= 8`, `gaji <= 8`, `risiko == sedang`, `status == pekerja`	IF `risiko == sedang` THEN Reksadana
R3	C4	C2 (cornerstone R1)	`umur > 30`, `umur >= 60`, `gaji > 15`, `gaji >= 20`, `status == pensiun`, `status != pekerja`	IF `status == pensiun` THEN Deposito
R4	C6	C2 (cornerstone R1)	`umur < 30`, `umur <= 22`, `gaji < 15`, `gaji <= 3`, `status == pelajar`, `status != pekerja`	IF `gaji <= 3` THEN Reksadana
R5	C8	C3 (cornerstone R2)	`umur > 25`, `umur >= 50`, `gaji > 8`, `gaji >= 40`	IF `gaji >= 40` THEN Obligasi
–	C9	C2 (cornerstone R1)	*kosong*	Conflict

Tabel 4: Difference list dan rule yang terbentuk

Dua hal penting dari tabel di atas:

- **R2** ditempel di bawah R1 (karena R1 adalah lastEval), tetapi yang dianulir adalah kesimpulan R0. Karena itu R2 dibandingkan dengan cornerstone R0, yang kosong.
- **R4** ditempel di bawah R3 (lastEval), tetapi dibandingkan dengan cornerstone R1 (C2), bukan cornerstone R3. Cornerstone R3 (C4) bernilai TRUE di R3 sehingga tidak pernah mencapai false-child R3.

6.4 Tree Akhir



Gambar 3: Tree akhir setelah skenario C1–C9 (kiri = F/false-child, kanan = T/true-child)

6.5 Uji Regresi Cornerstone

Setelah semua rule dibuat, setiap cornerstone ditelusuri ulang. Semuanya tetap berakhir di node yang dibuat untuknya, sehingga penambahan rule tidak merusak pengetahuan lama.

Cornerstone	Jalur pada tree akhir	Output	Sesuai
C2 (R1)	R0(T), R1(T), R3(F), R4(F)	Saham	Ya
C3 (R2)	R0(T), R1(F), R2(T), R5(F)	Reksadana	Ya
C4 (R3)	R0(T), R1(T), R3(T)	Deposito	Ya
C6 (R4)	R0(T), R1(T), R3(F), R4(T)	Reksadana	Ya
C8 (R5)	R0(T), R1(F), R2(T), R5(T)	Obligasi	Ya

Tabel 5: Uji regresi cornerstone

6.6 Penanganan Conflict

Kasus C9 identik dengan cornerstone R1 (C2), tetapi pakar meminta kesimpulan *Obligasi*. Tidak ada kondisi yang dapat membedakan keduanya, sehingga difference list kosong. Sistem menampilkan peringatan dan tidak mengubah rule lama. Pakar dapat membatalkan koreksi, atau menambah atribut baru, misalnya `tanggungan` (numerik) dengan nilai 2 untuk C9. Karena C2 tidak memiliki nilai `tanggungan`, kandidat `tanggungan >= 2` dan `tanggungan <= 2` bernilai FALSE untuk C2 dan dapat dipakai sebagai kondisi rule baru. Alur ini diuji otomatis pada `tests/test_cli.py`.

6.7 Edge Case

Situasi	Perilaku sistem
Input numerik bukan angka, di luar rentang, atau kategori di luar daftar	Ditolak dengan pesan kesalahan; pada CLI ditanyakan ulang.
Atribut belum diisi	Ditolak; semua atribut wajib diisi.
Pakar memilih kondisi yang juga dipenuhi cornerstone	Ditolak karena cornerstone ikut teranulir.
Kondisi kustom tidak dipenuhi kasus baru	Ditolak.
Kesimpulan koreksi sama dengan output sistem	Dianggap setuju, tidak ada rule baru.
Kesimpulan diketik dengan ejaan berbeda (“saham ”)	Dinormalisasi menjadi kesimpulan yang sudah ada (“Saham”).
Difference list kosong	Conflict: batal atau tambah atribut (lihat subbab sebelumnya).
Urutan rule (C7)	R3 bernilai TRUE sehingga R4 di false-child R3 tidak dievaluasi. Ini perilaku yang benar pada SCRDR.
Kesimpulan default diubah	Aman karena root tidak memiliki cornerstone.

Situasi	Perilaku sistem

Tabel 6: Penanganan edge case

6.8 Pengujian Otomatis

Proyek dilengkapi 52 unit test (pytest) yang seluruhnya lulus. Cakupannya:

- evaluasi kondisi numerik dan kategorikal, termasuk atribut yang tidak ada;
- pembangkitan difference list (dengan dan tanpa cornerstone, nilai sama, atribut baru);
- inferensi dan penempatan node pada true-child maupun false-child;
- validasi penambahan rule dan normalisasi kesimpulan;
- simpan, muat, dan reset KB serta validasi schema domain;
- skenario C1–C9 lengkap dengan bentuk tree akhir dan uji regresi;
- smoke test antarmuka Streamlit dan alur CLI (termasuk conflict dan rekaman log).

7 Kesimpulan dan Saran

7.1 Kesimpulan

Sistem pakar berbasis SCRDR berhasil dibangun sesuai spesifikasi. KB dimulai dari satu rule default dan berkembang melalui koreksi pakar. Setiap rule baru adalah pengecualian bagi kesimpulan yang salah, dengan kondisi yang dipilih dari difference list terhadap cornerstone. Engine bersifat generik terhadap domain dan mendukung atribut numerik maupun kategorikal. Pengujian skenario menunjukkan tidak ada regresi pada cornerstone lama.

7.2 Saran

- Mendukung *Multiple Classification RDR* agar satu kasus dapat memiliki lebih dari satu kesimpulan.
- Menyimpan kasus yang disetujui di root sebagai cornerstone tambahan, agar rule pengecualian pertama tidak mengubah hasil kasus-kasus tersebut.
- Menyediakan nilai atribut baru untuk cornerstone lama saat atribut ditambahkan.

8 Pembagian Kerja

Anggota	Tugas
<Nama Anggota 1> (13523065)	...
<Nama Anggota 2> (13523117)	...

Lampiran

- Repositori: <tautan repositori>
- Contoh rekaman sesi CLI: `docs/contoh_sesi_cli.txt`